

Reviewer 1

Novelty: 1 (not novel at all - there are many existing ideas that are the same)

Rationale: I find this idea is extremely similar to "GenDec: A robust generative Question-decomposition method for Multi-hop reasoning" by Wu et al. (2024). Link: <https://arxiv.org/html/2402.11166v1>

Feasibility: 8 (Highly Feasible: Straightforward to implement the idea and run all the experiments.)

Rationale: Technically, this idea can be quickly re-produced based on the aforementioned paper. Though the motivations and evaluations are different from the existing work, it shouldn't take too long to figure them out.

Expected Effectiveness: 3 (Low Effectiveness: The idea might work in some special scenarios but you don't expect it to work in general.)

Rationale: Given that the idea is too similar to an existing one, the author may need to create a new but related idea as a follow-up study of the aforementioned paper. This idea does have a different motivation from the aforementioned one, so it uses different evaluation methods, though.

Excitement: 2

Rationale: Reviewers may argue the originality and novelty of this idea if it's submitted to a venue. They may not find it exciting, either.

Overall Score: 1 (Critically flawed, trivial, or wrong, would be a waste of students' time to work on it)

Rationale: The students should probably think one-step-further of the existing study and they may eventually find a way to improve the existing system.

Confidence: 5 (You are absolutely certain that the evaluation is correct and very familiar with the relevant literature)

Reviewer 2

Novelty: 6 (reasonably novel - there are some notable differences from existing ideas and probably enough to turn into a new paper)

Rationale: Query decomposition and RAG separately are well studied, if there is no existing work that combines both (which I'm not aware of), then it's reasonably novel.

Feasibility: 10 (Easy: The whole proposed project can be quickly executed within a few days without requiring advanced technical skills.)

Rationale: It's just a series of prompting which should be easy for a CS PhD student.

Expected Effectiveness: 8 (Probably Effective: The idea should offer some significant improvement over current methods on the relevant benchmarks.)

Rationale: This method involves multiple fine-grained retrieval operations and should naturally outperform existing retrieval methods without decomposition.

Excitement: 6 (Learning positive: exciting enough to be accepted at a major AI conference, but still has some weaknesses or somewhat incremental)

Rationale: Although I believe in the effectiveness of the proposed method, the high latency compared to baselines is a concern—training an end-to-end model to reduce latency might be a good add-on.

Overall Score: 7 (Good idea, would be accepted by major AI conferences)

Rationale: This is a good idea. If there is no identical existing work and the authors conduct comprehensive experiments, it would be a good paper.

Confidence: 4 (You are confident but not absolutely certain that the evaluation is correct)

Reviewer 3

Novelty: 5 (somewhat novel - there are differences from existing ideas but not enough to turn into a new paper)

Rationale: The idea aims to tackle a question by breaking it down and solving it one by one with RAG. But it seems to be a more specialized way of CoT with RAG.

Feasibility: 5 (Moderately feasible: It can probably be executed within the given time frame but would require careful planning, efficient use of APIs or some advanced computational strategies to overcome the limited GPU resources, and would require some modifications to the original proposal to make it work.)

Rationale: The idea assumes a question can be broken down into subquestions where each subquestion is independent of the others. In cases where they are not independent, the method might suffer from issues or inefficiency. But maybe the distribution of these questions is more like a long tail and predominantly questions that can be easily broken down. And is there a case where the question is high-level mathematics and difficult to the point where it breaks down into a non-linear scale of the question text token?

Expected Effectiveness: 5 (Somewhat ineffective: There might be some chance that the proposed idea can work better than existing baselines but the improvement will be marginal or inconsistent.)

Rationale: The main question is how the sub-questions are created. We can break the question into conditioned parts from $p(q_0|q_0, \dots, q_n) \dots p(q_n|q_0, \dots, q_{n-1})$ where we assume them to be dependent, or we can use LLM to reason about their dependency. We can also ask the question by asking leveled sub-questions like "where is this person from" into "which country is this person from", "which city is this person from", "which district is this person from". The concern is that different methods might affect the performance differently.

Excitement: 6 (Learning positive: exciting enough to be accepted at a major AI conference, but still has some weaknesses or somewhat incremental)

Rationale: The idea seems exciting as it prevents LLM from shortcutting the question and hallucinating. But it needs more method formulation on how the question should be broken down. The very baseline implementation will just degrade to a CoT reasoning with RAG for each step. Because this could just be a subset of CoT methods in some sense.

Overall Score: 6 (Marginally above the acceptance threshold of major AI conferences)

Rationale: I believe there could be more comparison with CoT as motivation. Why should this be better with prompting the model step by step using RAG, and why are they different? And for problem formulation, it would be great if we can list more edgy examples of how questions can be divided to help pilot the prompting methods.

Confidence: 4 (You are confident but not absolutely certain that the evaluation is correct)

U Example Idea: Semantic Divergence Minimization: Reducing Hallucinations in Large Language Models through Iterative Concept Grounding

Semantic Divergence Minimization: Reducing Hallucinations in Large Language Models through Iterative Concept Grounding (Part 1)

1. Problem Statement: Large language models often generate hallucinations by diverging from the core semantic content of the input, especially in complex reasoning tasks. This problem undermines the reliability and trustworthiness of LLMs in critical applications that require accurate and factual responses.

2. Motivation: Current approaches like chain-of-thought prompting focus on generating intermediate steps but do not explicitly constrain semantic drift. By continuously grounding generated content to the original semantic space of the input, we can reduce hallucinations while preserving reasoning capabilities. This method leverages the LLM's own ability to extract and compare semantic concepts, creating a self-correcting mechanism that does not require external knowledge bases or complex architectures.

3. Proposed Method: We introduce Semantic Divergence Minimization (SDM) prompting. For each reasoning step, we prompt the model to:

1. Generate a candidate next step.
2. Extract key semantic concepts from the original input.
3. Measure semantic similarity between the candidate step and extracted concepts.
4. If similarity is below a threshold, regenerate the step with explicit instructions to incorporate more relevant concepts.
5. Repeat until convergence or maximum iterations.

This creates a semantic 'gravity well' that keeps reasoning tethered to the input's conceptual core.

Semantic Divergence Minimization: Reducing Hallucinations in Large Language Models through Iterative Concept Grounding (Part 2)

4. Step-by-Step Experiment Plan:

1. Dataset Preparation:

- Use two datasets: HotpotQA for multi-hop reasoning and GSM8K for complex math word problems.
- For HotpotQA, utilize the dev set (7,405 questions).
- For GSM8K, employ the test set (1,319 problems).

2. Baseline Implementation:

- Implement two baselines:
 - Standard prompting: directly asking the model to answer the question.
 - Chain-of-thought (CoT) prompting: asking the model to show its work step-by-step before giving the final answer.

3. SDM Implementation:

- Implement the SDM method with the following sub-steps for each reasoning iteration:
 - Generate next step.
 - Extract key concepts from input.
 - Measure semantic similarity.
 - Regenerate if below threshold.
 - Repeat until convergence or maximum iterations.

4. Prompt Engineering:

- Design prompts for each step of SDM. For example:
 - "Generate the next step in solving this problem:"
 - "Extract key concepts from the original question:"
 - "Rate the semantic similarity between these concepts and the generated step on a scale of 0-10:"
 - "Regenerate the step, focusing more on these key concepts:"

5. Hyperparameter Tuning:

- Experiment with different similarity thresholds (e.g., 5, 6, 7 out of 10) and maximum iteration limits (e.g., 3, 5, 7) to find the optimal balance between performance and computational cost.

6. Model Selection:

- Use GPT-4 as the primary model due to its advanced reasoning capabilities.
- Test GPT-3.5-turbo for comparison.

7. Evaluation:

- For HotpotQA, use the official evaluation script to compute Exact Match (EM) and F1 scores.
- For GSM8K, use accuracy as the metric.
- Manually review a subset of responses to assess qualitative improvements in reasoning and reduction of hallucinations.

8. Analysis:

- Compare SDM results with baselines.
- Analyze the correlation between semantic similarity scores and answer correctness.
- Examine cases where SDM significantly improves or fails to improve over baselines.

Semantic Divergence Minimization: Reducing Hallucinations in Large Language Models through Iterative Concept Grounding (Part 3)

5. Test Case Examples:

- **Example 1 (HotpotQA):** Input: Who was born first, Alec Guinness or Billie Holiday?
Baseline Output: Alec Guinness was born first. He was born on April 2, 1914, while Billie Holiday was born on April 7, 1915.
SDM Output: The SDM method provides a more structured and transparent reasoning process, reducing the risk of hallucination by grounding each step in the key concepts (birth dates) from the original question.
- **Example 2 (GSM8K):** Input: Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and sells the rest at the farmers' market daily for \$2 per egg. How much money does she make every week?
Baseline Output: Let's solve this step by step: Janet makes \$182 every week.
SDM Output: The SDM method not only provides a clear step-by-step solution but also explicitly verifies that all key concepts from the original question are incorporated, reducing the risk of introducing irrelevant information or hallucinating facts.

6. Fallback Plan: If the proposed SDM method does not significantly outperform baselines, we can pivot the project in several ways. First, we could conduct an in-depth analysis of where and why SDM fails, potentially uncovering insights about LLM reasoning processes. We might find that SDM works better for certain types of questions or reasoning tasks, which could lead to a more nuanced application of the method. Second, we could explore variations of SDM, such as using different prompts for concept extraction or similarity measurement, or incorporating a dynamic threshold that adjusts based on the complexity of the question. Third, we could combine SDM with other prompting techniques like chain-of-thought or self-consistency to create a hybrid approach. Finally, if the semantic grounding aspect proves challenging, we could shift focus to analyzing how LLMs interpret and maintain semantic consistency throughout multi-step reasoning, which could provide valuable insights for future work on reducing hallucinations.